Course Materials for NLP Term Project: The UnigramLM Tokenization Algorithm: A Worked Example

Prof. Moon-Ryul Jung
Dept of Art & Technology
Sogang University

1 What "SentencePiece" Means

A clarification before we begin: **SentencePiece is the name of a software library** developed by Google [1], not the name of a specific algorithm. The library supports two distinct tokenization algorithms:

- `-model_type=bpe`: implements Byte-Pair Encoding [3].
- `-model_type=unigram`: implements the UnigramLM algorithm of Kudo [2].

When a paper says "trained with SentencePiece," it is usually ambiguous about which algorithm was used. However, the **default** mode in the SentencePiece library is `unigram`, and this mode is also the more distinctive contribution of the library, since BPE itself was developed earlier and independently.

XLM-R, mBERT (in some configurations), NLLB, T5, and Glot500 all use SentencePiece in UnigramLM mode for their pretraining.

2 Setup: The Input Corpus

The input to UnigramLM training is a set of pre-tokens with their corpus frequencies:

$$\mathcal{W} = \{(w_i, c_i)\} \tag{1}$$

A pre-token is usually a whitespace-separated word (with a special boundary marker like ␣ prepended), and c_i is the number of times w_i appears in the corpus. The pre-token frequencies c_i are essential input — they tell the algorithm how to weight each pre-token's contribution to the corpus likelihood.

For our small example, the input corpus is:

$$\mathcal{W} = \{(\text{low}, 5),\ (\text{lower}, 2),\ (\text{newest}, 6),\ (\text{widest}, 3)\} \tag{2}$$

So the corpus contains four distinct pre-tokens with a total of $5 + 2 + 6 + 3 = 16$ pre-token occurrences.

2.1 How $\mathcal{W}$ Enters the Algorithm

Unlike BPE, where the pre-token frequencies enter directly as multiplicative weights on pair counts ($\text{freq}(a,b) = \sum_i c_i \cdot \#_i(a,b)$), UnigramLM uses $\mathcal{W}$ in two distinct stages:

Stage 1 — Seed vocabulary construction. The character stream of the corpus is scanned to find frequent substrings, which become candidate tokens in the seed vocabulary V_{init}. Each pre-token w_i contributes its character substrings c_i times (since the corpus contains c_i copies of w_i). So pre-token frequencies indirectly influence which substrings become candidates, but only via the character-stream view of the corpus.

Stage 2 — Estimating token probabilities. With V_{init} fixed, the algorithm needs to estimate a probability for each token. Before stating the optimization problem, we need to set up the notation carefully.

The parameters: θ. The unknown quantities to be estimated are the token probabilities. Collecting them into one object:

$$\theta = \{p_\theta(t)\}_{t \in V_{\text{init}}}, \quad \text{with} \quad p_\theta(t) \in [0,1] \quad \text{and} \quad \sum_{t \in V_{\text{init}}} p_\theta(t) = 1 \tag{3}$$

So θ is a vector of $|V_{\text{init}}|$ probabilities (one per token), constrained to sum to 1. These are the parameters of the multinomial unigram model. The notation $p_\theta(t)$ (rather than just $p(t)$) makes explicit that this is the probability of token t *under parameter setting* θ.

The latent variables: segmentations as cut locations. A pre-token w_i is a string of characters of length $|w_i|$. A segmentation of w_i is determined entirely by the set of **cut positions** between characters — positions in $\{1, 2, \ldots, |w_i| - 1\}$ where the string is split into substrings.

For example, for $w_i =$ "newest" (length 6, so cut positions could be any subset of $\{1,2,3,4,5\}$):

- Cut positions $\{\}$ (no cuts): $\tau = (\text{newest})$.
- Cut positions $\{3\}$: $\tau = (\text{new}, \text{est})$.
- Cut positions $\{1,2,3,4,5\}$ (all cuts): $\tau = (\text{n}, \text{e}, \text{w}, \text{e}, \text{s}, \text{t})$.
- Cut positions $\{1,2,3\}$: $\tau = (\text{n}, \text{e}, \text{w}, \text{est})$.

The constraint is that every resulting substring must be a valid token in V_{init}. So the set of valid segmentations of w_i is:

$$\mathcal{T}_{V_{\text{init}}}(w_i) = \{S \subseteq \{1, \ldots, |w_i| - 1\} : \text{every substring induced by } S \text{ is in } V_{\text{init}}\} \tag{4}$$

The latent variable for pre-token w_i is its segmentation $\tau_i \in \mathcal{T}_{V_{\text{init}}}(w_i)$ — equivalently, its set of cut positions. This is what the corpus does **not** tell us: we see the surface form "newest" but we do not see whether the writer "meant" it as (new, est) or as $(\text{n}, \text{e}, \text{w}, \text{e}, \text{s}, \text{t})$. The cut positions are hidden.

Observed vs. latent. To make the distinction crisp:

- **Observed**: the corpus $\mathcal{W} = \{(w_i, c_i)\}$ — which pre-tokens appeared and how often.
- **Latent (hidden)**: the segmentations $\{\tau_i\}$ — which cut positions were "actually" used. There are exponentially many possible cut-position sets per pre-token (up to $2^{|w_i|-1}$), most of which yield valid segmentations under a large V_{init}.
- **Parameters to estimate**: $\theta = \{p_\theta(t)\}_{t \in V_{\text{init}}}$.

The probabilistic model. The model is generative: it imagines that each pre-token w_i was produced by first sampling a segmentation $\tau \in \mathcal{T}_{V_{\text{init}}}(w_i)$ (the latent variable), then concatenating its tokens to form the surface string. Under the unigram independence assumption, the joint probability of pre-token w_i and segmentation τ is:

$$p_\theta(w_i, \tau) = p_\theta(\tau) = \prod_{t_k \in \tau} p_\theta(t_k) \quad \text{(since } w_i \text{ is deterministic given } \tau\text{)} \tag{5}$$

The marginal probability of observing just the pre-token w_i (without observing τ) is obtained by summing over all possible latent segmentations:

$$\boxed{p_\theta(w_i) = \sum_{\tau \in \mathcal{T}_{V_{\text{init}}}(w_i)} p_\theta(\tau) = \sum_{\tau \in \mathcal{T}_{V_{\text{init}}}(w_i)} \prod_{t_k \in \tau} p_\theta(t_k)} \tag{6}$$

This $p_\theta(w_i)$ is a *derived* quantity — it is computed from the parameters θ, not a parameter itself. The parameters of the model are the per-token probabilities in θ; the marginal $p_\theta(w_i)$ is what the model assigns to each observed pre-token after marginalizing out the unobserved cut positions.

The optimization problem. We want to choose θ to maximize the corpus log-likelihood — the probability that the model assigns to the observed corpus, given that the segmentations were marginalized out:

$$\theta^* = \arg\max_\theta \mathcal{L}(\theta; \mathcal{W}), \quad \mathcal{L}(\theta; \mathcal{W}) = \sum_i c_i \log p_\theta(w_i) \tag{7}$$

This is the standard maximum likelihood estimation (MLE) objective, with one twist: the log is around a sum (the marginal $p_\theta(w_i)$ is a sum over segmentations), so the log-likelihood has no closed-form maximum in θ.

Why MLE is hard here: the latent-variable problem. If we could observe the true cut positions for each pre-token, MLE would be trivial:

$$p^*_\theta(t) = \frac{\text{count of } t \text{ across all observed segmentations}}{\text{total token count}} \tag{8}$$

This is just empirical frequency, the closed-form MLE for a multinomial distribution. But because cut positions are latent, we do not have those counts. We only have surface forms w_i, from which any of many possible cut-position sets could have produced.

The standard solution: EM. The Expectation-Maximization (EM) algorithm of Dempster, Laird, and Rubin [4] handles exactly this latent-variable MLE situation. Instead of using observed token counts (which we do not have), it uses **expected token counts** under the current model. The algorithm is iterative:

1. Start with some initial parameter setting $\theta^{(0)}$ (e.g., uniform $p_\theta(t) = 1/|V_{\text{init}}|$).

2. **E-step**: Under the current $\theta^{(k)}$, compute the posterior distribution over the latent cut positions for each pre-token: $p_{\theta^{(k)}}(\tau \mid w_i)$. From this, compute the expected count $E_{\theta^{(k)}}[\#t]$ of each token t.

3. **M-step**: Update parameters by normalizing the expected counts:

$$p_{\theta^{(k+1)}}(t) = \frac{E_{\theta^{(k)}}[\#t]}{\sum_{t' \in V_{\text{init}}} E_{\theta^{(k)}}[\#t']} \tag{9}$$

   This is the same closed-form MLE formula as before — but applied to expected counts rather than observed counts.

4. Repeat until convergence.

EM is guaranteed to non-decrease the corpus log-likelihood $\mathcal{L}(\theta; \mathcal{W})$ at each iteration. It converges to a local maximum of $\mathcal{L}$.

Aggregating expected counts across the corpus (this is where c_i enters explicitly):

$$E_\theta[\#t] = \sum_i c_i \cdot E_\theta[\#t \mid w_i] \tag{10}$$

A pre-token appearing 6 times contributes 6 times as much to each token's expected count as a pre-token appearing once. The per-pre-token expected count $E_\theta[\#t \mid w_i]$ involves summing over all valid cut-position sets weighted by their posterior probability, and is computed efficiently by the forward-backward algorithm on the segmentation lattice (details in Section 6).

Summary of the relationship. $\mathcal{W}$ is required input to UnigramLM, but it enters the algorithm via two indirect channels:

- Indirectly through the character stream (Stage 1, seed construction).
- Indirectly through the EM aggregation of expected counts weighted by c_i (Stage 2).

Compared to BPE, which directly weights merge decisions by c_i, UnigramLM's use of $\mathcal{W}$ is mediated by the latent-variable structure of segmentations and the EM machinery for handling it. We will see both channels at work in the walkthrough.

In our example, we will track how the vocabulary V, the parameters θ (token probabilities), and the tokenization of "newest" evolve across the training process.

2.2 The Training Budget

As with BPE, the training budget for UnigramLM is specified by a target vocabulary size V_{target}. However, the relationship between budget and iterations is fundamentally different from BPE:

BPE (bottom-up): Each iteration adds exactly one token. To reach V_{target}, the algorithm runs for $N_{\text{merge}} = V_{\text{target}} - |V_0|$ iterations.

UnigramLM (top-down): Each iteration removes some fraction of the existing vocabulary. The algorithm starts from a large seed vocabulary V_{init} and prunes downward to V_{target}. The number of pruning iterations depends on the shrinkage factor $\eta \in (0, 1)$ (typically $\eta \approx 0.8$).

The number of UnigramLM iterations is approximately:

$$N_{\text{iter}} \approx \frac{\log(V_{\text{target}}/V_{\text{init}})}{\log \eta} \tag{11}$$

For example, if $V_{\text{init}} = 100$ and $V_{\text{target}} = 14$ and $\eta = 0.8$, then $N_{\text{iter}} \approx \log(0.14)/\log(0.8) \approx 8.8$, so about 9 iterations.

In practice, the exact iteration count is less interesting than the convergence behavior, because each pruning iteration also involves an inner EM loop to refine token probabilities. The budget is effectively governed by V_{target} and the algorithm runs until $|V_t| \leq V_{\text{target}}$.

For our example. We set $V_{\text{target}} = 14$ (matching the BPE example) and $V_{\text{init}} = 25$ (a small seed that we will define explicitly). We will trace the algorithm through pruning down to size 14.

3 Overview of the Training Procedure

Before diving into details, here is the overall structure of UnigramLM training so that the reader knows what is being built up:

1. **Define a probabilistic model** over segmentations (Section 4). This model is what we will fit to the corpus.
2. **Initialize**: build a large seed vocabulary V_{init} and assign initial probabilities (Section 5).
3. **Iterate** the following two steps until $|V| \leq V_{\text{target}}$ (Sections 6–7):
   - **Inner EM loop**: re-estimate token probabilities $p(t)$ that maximize corpus likelihood, holding the current vocabulary fixed.
   - **Pruning step**: remove the bottom $1 - \eta$ fraction of tokens (those that contribute least to corpus likelihood). Single characters are protected.
4. **Output**: the final vocabulary V_{target} with token probabilities, used at inference via Viterbi (Section 8).

The word "iteration" throughout this tutorial refers to one outer step of the loop (one EM round + one pruning step). The number of iterations is determined by V_{init}, V_{target}, and the shrinkage factor η, not specified by the user directly.

We now describe each piece in detail, starting with the probabilistic model.

4 The Probabilistic Model

UnigramLM is a real generative model, not a procedure. The model is defined **given** a vocabulary V (a set of allowed token strings) and parameters $\theta = \{p_\theta(t)\}_{t \in V}$ (probabilities, one per token).

Important note about notation. V and θ play very different roles in UnigramLM:

- V **is a structural choice** — which tokens are allowed at all. V is changed by the outer pruning loop (Sections 6–7), where tokens are removed from V across iterations.
- θ **is a continuous parameter vector** — the probability assigned to each token in the current V. θ is optimized by the inner EM loop within each outer iteration, holding the current V fixed.

In this section, we describe the model and its likelihood **for a given** V, without yet worrying about how V is chosen. Later sections describe how V shrinks from V_{init} to V_{target} across pruning iterations.

4.1 The Generative Story

Given a vocabulary V and parameters $\theta = \{p_\theta(t)\}_{t \in V}$ with $\sum_{t \in V} p_\theta(t) = 1$, the probability of a segmentation $\tau = (t_1, \ldots, t_K)$ is:

$$p_\theta(\tau) = \prod_{k=1}^{K} p_\theta(t_k) \tag{12}$$

(Unigram independence assumption: tokens drawn independently.)

The probability of a pre-token w_i (its surface string) is the marginal over all valid segmentations under the current V:

$$p_\theta(w_i \mid V) = \sum_{\tau \in \mathcal{T}_V(w_i)} p_\theta(\tau) \tag{13}$$

where $\mathcal{T}_V(w_i)$ is the set of all valid segmentations of w_i using only tokens from V. Note the conditioning on V: the marginal depends on which tokens are allowed.

4.2 The Likelihood Given V

For a fixed vocabulary V, the corpus log-likelihood as a function of the parameters θ is:

$$\mathcal{L}(\theta \mid V; \mathcal{W}) = \sum_i c_i \log p_\theta(w_i \mid V) \tag{14}$$

The notation $\mathcal{L}(\theta \mid V; \mathcal{W})$ reads: "the likelihood is a function of θ, given that the vocabulary is fixed at V, evaluated on data $\mathcal{W}$." This is the objective that the inner EM loop maximizes. Notice the c_i — pre-token frequencies enter the objective here, with a pre-token appearing 6 times contributing 6 copies of $\log p_\theta(w_i \mid V)$ to the total.

4.3 What "UnigramLM Learning" Actually Optimizes

Strictly speaking, UnigramLM training is a two-level optimization:

- **Inner loop (EM)**: Holding V fixed, find the parameters $\theta^*(V)$ that maximize $\mathcal{L}(\theta \mid V; \mathcal{W})$.
- **Outer loop (pruning)**: Search over vocabularies $V \subseteq V_{\text{init}}$ with $|V| \leq V_{\text{target}}$ to maximize $\mathcal{L}(\theta^*(V) \mid V; \mathcal{W})$.

The outer problem is combinatorial: there are exponentially many subsets of V_{init} to consider. UnigramLM does not solve it exactly; instead, it uses a **greedy heuristic**: start with V_{init}, run EM to find $\theta^*(V_{\text{init}})$, then identify the least-useful tokens (smallest contribution to $\mathcal{L}$) and remove them. Repeat. This greedy pruning is not guaranteed to find the global optimum, but it works well in practice.

So when we write "UnigramLM finds V and θ," we mean: it greedily searches for a V of size at most V_{target} together with parameters $\theta^*(V)$ for that V, approximately maximizing the marginal likelihood. The combined objective is:

$$(V^*, \theta^*) \approx \arg \max_{V \subseteq V_{\text{init}}, |V| \leq V_{\text{target}}} \max_\theta \mathcal{L}(\theta \mid V; \mathcal{W}) \tag{15}$$

The outer arg max is solved greedily; the inner max is solved by EM (approximately, since EM finds a local maximum).

For the rest of this tutorial, we will mostly drop the conditioning notation and write $p_\theta(w_i)$ rather than $p_\theta(w_i \mid V)$, and $\mathcal{L}(\theta; \mathcal{W})$ rather than $\mathcal{L}(\theta \mid V; \mathcal{W})$ — but with the understanding that there is always an implicit "given the current V." The current V is the V_t at the start of iteration t of the outer pruning loop.

5 Iteration 0: Initial State

5.1 Building the Seed Vocabulary V_{init}

UnigramLM starts from a large seed vocabulary containing many candidate substrings of the corpus. In SentencePiece, the seed is typically built by:

1. Including all single characters from the corpus (mandatory — ensures every pre-token can be segmented).
2. Including additional candidate multi-character substrings, often the most frequent substrings up to some maximum length (default 16 characters in SentencePiece).

The substring-frequency computation uses an Enhanced Suffix Array (ESA) over the concatenated corpus text. In this corpus view, each pre-token w_i appears c_i times — so the substrings of high-frequency pre-tokens are over-represented in the substring statistics. This is the first place where $\mathcal{W}$'s structure enters.

For our example, let us take a concrete seed. We include all 10 single characters plus 15 frequent multi-character substrings:

$$V_{\text{init}} = \{\underbrace{\text{l}, \text{o}, \text{w}, \text{e}, \text{r}, \text{n}, \text{s}, \text{t}, \text{i}, \text{d}}_{\text{single characters (10)}}, \underbrace{\text{lo}, \text{ow}, \text{we}, \text{er}, \text{ne}, \text{es}, \text{st}, \text{id}, \text{de}, \text{wi}, \text{est}, \text{new}, \text{low}, \text{ower}, \text{idest}}_{\text{candidate multi-character tokens (15)}}\} \tag{16}$$

So $|V_{\text{init}}| = 25$.

5.2 Initial Probabilities

The algorithm initializes token probabilities uniformly:

$$p^{(0)}(w) = \frac{1}{|V_{\text{init}}|} = \frac{1}{25} = 0.04 \quad \text{for all } w \in V_{\text{init}} \tag{17}$$

This uniform initialization is a starting point; the EM loop in iteration 1 will refine these probabilities based on the corpus.

5.3 Initial Segmentation Possibilities for "newest"

Here is where UnigramLM differs fundamentally from BPE. UnigramLM does not commit to a single segmentation. Instead, it considers the full posterior distribution over segmentations.

For "newest" under V_{init}, the valid segmentations include:

- $(\text{n}, \text{e}, \text{w}, \text{e}, \text{s}, \text{t})$ — all characters
- $(\text{n}, \text{e}, \text{w}, \text{es}, \text{t})$ — using es
- $(\text{n}, \text{e}, \text{w}, \text{e}, \text{st})$ — using st
- $(\text{n}, \text{e}, \text{w}, \text{est})$ — using est
- $(\text{n}, \text{e}, \text{we}, \text{s}, \text{t})$ — using we

- $(\mathrm{n}, \mathrm{e}, \mathrm{we}, \mathrm{st})$ — using we and st
- $(\mathrm{ne}, \mathrm{w}, \mathrm{e}, \mathrm{s}, \mathrm{t})$ — using ne
- $(\mathrm{ne}, \mathrm{w}, \mathrm{es}, \mathrm{t})$ — using ne and es
- $(\mathrm{ne}, \mathrm{w}, \mathrm{e}, \mathrm{st})$ — using ne and st
- $(\mathrm{ne}, \mathrm{w}, \mathrm{est})$ — using ne and est
- $(\mathrm{new}, \mathrm{e}, \mathrm{s}, \mathrm{t})$ — using new
- $(\mathrm{new}, \mathrm{es}, \mathrm{t})$ — using new and es
- $(\mathrm{new}, \mathrm{e}, \mathrm{st})$ — using new and st
- $(\mathrm{new}, \mathrm{est})$ — using new and est

There are 14 valid segmentations of "newest" under V_{init}. Each has a probability:

$$p(\tau) = \prod_k p^{(0)}(t_k) = (0.04)^{|\tau|} \tag{18}$$

For instance, $p((\mathrm{new}, \mathrm{est})) = 0.04^2 = 0.0016$, while $p((\mathrm{n}, \mathrm{e}, \mathrm{w}, \mathrm{e}, \mathrm{s}, \mathrm{t})) = 0.04^6 \approx 4.1 \times 10^{-9}$. The probability of the string "newest" is the sum:

$$p(\text{“newest”}) = \sum_{\tau \in \mathcal{T}_{V_{\text{init}}}(\text{newest})} p(\tau) \tag{19}$$

6 Iteration 1: EM Step Followed by Pruning

6.1 Inner EM Loop: Refining Probabilities

Holding $V_1 = V_{\text{init}}$ fixed, EM updates the token probability function $p : V_1 \to [0, 1]$, where $p(t)$ is the probability assigned to each token $t \in V_1$. The initial uniform values $p^{(0)}(t) = 1/|V_{\text{init}}| = 0.04$ (from Section 5) are refined to new values $p^{(1)}(t)$ that fit the model (parameterized by p) to the observed corpus $\mathcal{W}$.

What does "fit the model parameters θ to the corpus $\mathcal{W}$" mean concretely? It means: choose θ to maximize the probability that the model would generate $\mathcal{W}$, given that the vocabulary V_1 is held fixed. By convention this is expressed in log-probability form (which is monotonic but numerically nicer):

$$\mathcal{L}(\theta \mid V_1; \mathcal{W}) = \sum_i c_i \log p_\theta(w_i \mid V_1) = \sum_i c_i \log \left(\sum_{\tau \in \mathcal{T}_{V_1}(w_i)} \prod_{t_k \in \tau} p_\theta(t_k) \right) \tag{20}$$

This quantity is the **corpus log-likelihood under the model** — a function of θ, given the fixed vocabulary V_1 and the data $\mathcal{W}$. Recall from Section 4 that $p_\theta(w_i \mid V_1)$ is the marginal probability of pre-token w_i summing over all valid segmentations under V_1. The log-likelihood says: a pre-token w_i that the model thinks is probable contributes a large (less negative) $\log p_\theta(w_i \mid V_1)$, weighted by how often w_i appears in the corpus (c_i). High $\mathcal{L}$ means the model assigns high probability to what the corpus actually contains.

Under uniform $\theta^{(0)}$ (every $p_\theta(t) = 1/|V_1|$), the model treats every token as equally likely. This is a poor fit: "newest" under uniform probabilities has many roughly-equal-probability segmentations, and the marginal $p_\theta(\text{newest} \mid V_1)$ is small because probability mass is spread thinly. After EM, tokens that frequently appear in high-probability segmentations of high-frequency pre-tokens (like est, which helps tokenize "newest" and "widest") get more probability mass, and the marginal $p_\theta(w_i \mid V_1)$ for high-frequency pre-tokens goes up. The corpus log-likelihood increases as a result.

The optimization goal: find

$$\theta^*(V_1) = \arg\max_\theta \mathcal{L}(\theta \mid V_1; \mathcal{W}) \quad \text{subject to} \quad \sum_{t \in V_1} p_\theta(t) = 1,\ p_\theta(t) \geq 0 \tag{21}$$

The notation $\theta^*(V_1)$ emphasizes that the optimal parameters depend on the current vocabulary V_1. When the outer pruning loop changes V at the next iteration, the optimal parameters change too — so EM is re-run.

This is a maximum likelihood estimation (MLE) problem with a complication: the segmentation τ of each pre-token w_i is a **latent** (unobserved) variable. We see w_i in the corpus but we do not see which cut positions produced it. The log-likelihood involves a sum inside a logarithm ($\log \sum_\tau \prod_k p_\theta(t_k)$), which has no closed-form maximum in θ.

EM is the standard algorithm for MLE with latent variables. The Expectation-Maximization (EM) algorithm of Dempster, Laird, and Rubin [4] handles exactly this situation: it iteratively maximizes a lower bound on $\mathcal{L}$ that is easier to compute than $\mathcal{L}$ itself. Each iteration alternates two steps:

- **E-step (Expectation)**: Using the current θ, compute the posterior distribution over the latent variable (the segmentation τ, i.e., the cut positions) for each pre-token. From this posterior, compute the *expected sufficient statistics* — here, the expected counts $E_\theta[\#t]$ of each token appearing in segmentations.
- **M-step (Maximization)**: Treating the expected counts as if they were observed counts, update θ to maximize a tractable surrogate likelihood. For multinomial distributions (which θ parameterizes), this update is just normalizing the expected counts.

Iterating these two steps is guaranteed to non-decrease $\mathcal{L}$ at each round and converges (typically in a few iterations) to a local maximum of $\mathcal{L}$.

E-step in detail. The E-step computes the expected count $E[\#t]$ for each token $t \in V_1$. Recall that $\#t$ denotes the random count of token t across the segmentations the corpus might have. The expectation is taken with respect to the posterior distribution over segmentations under the current parameters p.

For one pre-token w_i, the expected count of token t given that w_i was the observation is:

$$E[\#t \mid w_i] = \sum_{\tau \in \mathcal{T}_{V_1}(w_i)} p(\tau \mid w_i) \cdot n_t(\tau) \tag{22}$$

where $p(\tau \mid w_i) = p(\tau)/p(w_i)$ is the posterior probability of segmentation τ given that we observed w_i, and $n_t(\tau)$ is the literal integer count of token t in segmentation τ. So $E[\#t \mid w_i]$ is the average count of t across possible segmentations of w_i, weighted by how plausible each segmentation is under the current model.

Computing $E[\#t \mid w_i]$ efficiently via forward-backward. Naively enumerating all $|\mathcal{T}_{V_1}(w_i)|$ segmentations is exponential in pre-token length. The forward-backward algorithm computes the expected counts in $O(|w_i|^2)$ time by exploiting the lattice structure of segmentations.

Define for each character position $j \in \{0, 1, \ldots, |w_i|\}$ in pre-token w_i:

$$\alpha_i(j) = \sum_{k: w_i[k:j] \in V_1} \alpha_i(k) \cdot p(w_i[k:j]) \quad \text{(forward)} \tag{23}$$

$$\beta_i(j) = \sum_{k: w_i[j:k] \in V_1} p(w_i[j:k]) \cdot \beta_i(k) \quad \text{(backward)} \tag{24}$$

with boundary conditions $\alpha_i(0) = 1$ and $\beta_i(|w_i|) = 1$. Intuitively, $\alpha_i(j)$ is the total probability of all partial segmentations covering characters 0 to j, and $\beta_i(j)$ is the total probability of all partial segmentations covering characters j to $|w_i|$. The marginal pre-token probability is then $p(w_i) = \alpha_i(|w_i|) = \beta_i(0)$.

For a specific token occurrence $t = w_i[a:b]$ (token t spans positions a to b in w_i), the expected count contribution is:

$$E[\#t \mid w_i, \text{at position } [a:b]] = \frac{\alpha_i(a) \cdot p(t) \cdot \beta_i(b)}{p(w_i)} \tag{25}$$

The numerator is the total probability of all complete segmentations of w_i that use token t specifically at positions $[a:b]$. The denominator $p(w_i)$ normalizes by the total probability of w_i, converting joint probabilities into a conditional (posterior) probability. So this expression is exactly the posterior probability that t is used at position $[a:b]$ in the segmentation of w_i — which is also the expected number of times t appears at that specific position (the indicator variable's expectation).

Summing over all positions $[a:b]$ where t could appear in w_i gives $E[\#t \mid w_i]$, the total expected count of t across all positions of w_i.

Aggregating across the corpus (this is where c_i enters explicitly):

$$\boxed{E[\#t] = \sum_i c_i \cdot E[\#t \mid w_i]} \tag{26}$$

The pre-token frequency c_i weights each pre-token's contribution. In our example:

- "low" (frequency 5) contributes $5 \cdot E[\#t \mid \text{low}]$ to each token's count.
- "newest" (frequency 6) contributes $6 \cdot E[\#t \mid \text{newest}]$ to each token's count.

So a token that helps segment "newest" efficiently gets a larger expected count than one that only helps segment "lower" (frequency 2), all else being equal.

M-step. Given the expected counts $E[\#t]$ from the E-step, update the probabilities by normalized maximum likelihood (the closed-form MLE solution for a multinomial distribution):

$$p^{(\text{new})}(t) = \frac{E[\#t]}{\sum_{t' \in V_1} E[\#t']} \tag{27}$$

Each new probability is the token's expected count divided by the total expected count, so $\sum_t p^{(\text{new})}(t) = 1$ by construction. This is exactly what you would do if $E[\#t]$ were observed counts from a real sample; EM's trick is to substitute expected counts (which we can compute) for the observed counts (which we cannot, because segmentations are latent).

After several EM iterations (typically converging in a few rounds), the probabilities settle into estimates that reflect each token's empirical utility in segmenting the corpus, weighted by pre-token frequencies c_i.

Token	$p(t)$	**Token**	$p(t)$
l	0.030	lo	0.090
o	0.020	ow	0.020
w	0.045	we	0.005
e	0.060	er	0.030
r	0.005	ne	0.020
n	0.010	es	0.110
s	0.020	st	0.110
t	0.020	id	0.005
i	0.010	de	0.005
d	0.005	wi	0.045
		est	0.120
		new	0.090
		low	0.105
		ower	0.020
		idest	0.000

Table 1: Token probabilities after EM convergence at iteration 1. These are illustrative values, hand-tuned to match the example.

Resulting probabilities after EM (illustrative, hand-tuned to match the example).
After EM converges with $V_1 = V_{\text{init}}$, the probabilities approximately become:

Tokens that appear frequently and contribute to high-probability segmentations of high-frequency pre-tokens get high probability. For instance, est has probability 0.12 because it helps segment both "newest" (frequency 6) and "widest" (frequency 3) efficiently. Tokens that almost never get used (e.g., idest, de, we) get tiny probabilities or near-zero.

Note how c_i shapes these probabilities: est gets credit for helping with $6 + 3 = 9$ total instances (weighting by c_i), while ower only gets credit for 2 instances (lower with frequency 2). This is why est ends up with much higher probability than ower, even though both are equally well-fit to one of the pre-tokens.

6.2 Pruning Step

After EM converges, the algorithm computes how much each token contributes to the corpus likelihood. For each candidate token t, the loss of removing t is approximately:

$$\Delta_t \approx E[\#t] \cdot \log p(t) \tag{28}$$

Tokens with the lowest $|\Delta_t|$ (least useful) are removed. Single characters are protected from pruning to ensure full coverage.

For our example, the lowest-contribution tokens are idest, we, de, wi, id, ower, er, ow, ne, st, lo (in order of increasing contribution). The algorithm removes the bottom $1 - \eta = 0.2$ fraction. With $|V_1| = 25$, that means about 5 tokens.

Tokens pruned at iteration 1: $\{\text{idest}, \text{we}, \text{de}, \text{id}, \text{wi}\}$.

Updated vocabulary:

$$\begin{aligned} V_2 &= V_{\text{init}} \setminus \{\text{idest}, \text{we}, \text{de}, \text{id}, \text{wi}\} \\ &= \{\text{l}, \text{o}, \text{w}, \text{e}, \text{r}, \text{n}, \text{s}, \text{t}, \text{i}, \text{d}, \\ &\quad \text{lo}, \text{ow}, \text{er}, \text{ne}, \text{es}, \text{st}, \\ &\quad \text{est}, \text{new}, \text{low}, \text{ower}\} \end{aligned} \tag{29}$$

with $|V_2| = 20$.

6.3 Posterior over "newest" Segmentations at End of Iteration 1

With the new V_2 and probabilities re-normalized (we omit re-normalization details for brevity), the valid segmentations of "newest" have changed.

Some segmentations are no longer valid (e.g., those using we); they have probability 0.

Other segmentations remain. For instance:

$$p((\text{new}, \text{est})) \propto p(\text{new}) \cdot p(\text{est}) = 0.090 \cdot 0.120 = 0.0108 \tag{30}$$

$$p((\text{ne}, \text{w}, \text{est})) \propto p(\text{ne}) \cdot p(\text{w}) \cdot p(\text{est}) = 0.020 \cdot 0.045 \cdot 0.120 = 0.000108 \tag{31}$$

$$p((\text{n}, \text{e}, \text{w}, \text{est})) \propto p(\text{n}) \cdot p(\text{e}) \cdot p(\text{w}) \cdot p(\text{est}) \tag{32}$$

$$= 0.010 \cdot 0.060 \cdot 0.045 \cdot 0.120 = 3.24 \times 10^{-6} \tag{33}$$

The posterior gives much higher mass to (new, est) than to the character-level segmentation.

Viterbi segmentation (if we were to commit). If we performed Viterbi decoding now, the most probable segmentation would be:

$$\tau^*_{\text{newest},V_2} = (\text{new}, \text{est}) \tag{34}$$

But UnigramLM does not commit at this stage. The posterior over all valid segmentations remains the operative object.

7 Iterations 2 and 3: Further Pruning

The algorithm iterates: EM converges with the new vocabulary to find new probabilities, then prunes the lowest-utility tokens. In both EM steps, the pre-token frequencies c_i continue to weight the aggregation:

$$E[\#t] = \sum_i c_i \cdot E[\#t \mid w_i] \tag{35}$$

This consistency is important — it means the algorithm's preference for tokens useful in high-frequency pre-tokens is maintained throughout pruning.

7.1 Iteration 2

After EM, low-utility tokens like ow, ower, lo, ne have small contributions. Prune the bottom $1 - \eta = 0.2$ fraction of $|V_2| = 20$, which is about 4 tokens.

Tokens pruned at iteration 2: {ower, ne, ow, lo}.

Updated vocabulary:

$$V_3 = \{\text{l}, \text{o}, \text{w}, \text{e}, \text{r}, \text{n}, \text{s}, \text{t}, \text{i}, \text{d}, \text{er}, \text{es}, \text{st}, \text{est}, \text{new}, \text{low}\} \tag{36}$$

with $|V_3| = 16$.

7.2 Iteration 3

After EM, more low-utility tokens are pruned. With V_3 at 16 tokens, we need to reach $V_{\text{target}} = 14$, so we prune 2 more.

Tokens pruned at iteration 3: $\{\text{st}, \text{er}\}$.

Updated vocabulary (final):

$$V_4 = \{\text{l}, \text{o}, \text{w}, \text{e}, \text{r}, \text{n}, \text{s}, \text{t}, \text{i}, \text{d}, \text{es}, \text{est}, \text{new}, \text{low}\} \tag{37}$$

with $|V_4| = 14 = V_{\text{target}}$. The training budget is exhausted. The algorithm terminates.

7.3 Final Probabilities

After EM converges with V_4, the final token probabilities (illustrative) might be:

Token	$p(t)$	**Token**	$p(t)$
l	0.05	i	0.04
o	0.05	d	0.04
w	0.08	es	0.10
e	0.08	est	0.15
r	0.04	new	0.10
n	0.05	low	0.12
s	0.05		
t	0.05		

Table 2: Final token probabilities at the end of training (illustrative).

8 Tokenization at Inference: Viterbi

Now that training is complete, we use UnigramLM to tokenize text at inference. **Viterbi decoding** finds the most probable segmentation under the final model.

At inference time, pre-token frequencies c_i no longer play any role. The algorithm tokenizes each input pre-token (or string) independently using only the vocabulary V_4 and the token probabilities p. The role of c_i was to shape these probabilities during training, but once training is complete, Viterbi treats each input string the same way regardless of how many times it appeared during training.

8.1 Tokenizing "newest"

We build the segmentation lattice for "newest" over V_4:

Nodes: $\{0, 1, 2, 3, 4, 5, 6\}$ (positions before character 0, between each pair, and after character 6).

Edges: An edge (i, j) exists if $\text{newest}[i : j] \in V_4$.

Listing the edges with their weights $-\log p$:

Viterbi computation. Let $d(j)$ be the shortest-path distance from node 0 to node j. Initialize $d(0) = 0$ and $d(j) = +\infty$ otherwise.

Edge	Substring	Probability	Weight $-\log p$
$(0,1)$	n	0.05	3.00
$(1,2)$	e	0.08	2.53
$(2,3)$	w	0.08	2.53
$(3,4)$	e	0.08	2.53
$(4,5)$	s	0.05	3.00
$(5,6)$	t	0.05	3.00
$(3,5)$	es	0.10	2.30
$(3,6)$	est	0.15	1.90
$(0,3)$	new	0.10	2.30

Table 3: Edges in the segmentation lattice for "newest" over V_4.

$$d(0) = 0 \tag{38}$$
$$d(1) = d(0) + \text{weight}(0,1) = 0 + 3.00 = 3.00 \tag{39}$$
$$d(2) = d(1) + \text{weight}(1,2) = 3.00 + 2.53 = 5.53 \tag{40}$$
$$d(3) = \min\{d(2) + \text{weight}(2,3),\ d(0) + \text{weight}(0,3)\} \tag{41}$$
$$= \min\{5.53 + 2.53,\ 0 + 2.30\} \tag{42}$$
$$= \min\{8.06,\ 2.30\} = 2.30 \quad \text{via edge } (0,3) \tag{43}$$
$$d(4) = d(3) + \text{weight}(3,4) = 2.30 + 2.53 = 4.83 \tag{44}$$
$$d(5) = \min\{d(4) + \text{weight}(4,5),\ d(3) + \text{weight}(3,5)\} \tag{45}$$
$$= \min\{4.83 + 3.00,\ 2.30 + 2.30\} \tag{46}$$
$$= \min\{7.83,\ 4.60\} = 4.60 \quad \text{via edge } (3,5) \tag{47}$$
$$d(6) = \min\{d(5) + \text{weight}(5,6),\ d(3) + \text{weight}(3,6)\} \tag{48}$$
$$= \min\{4.60 + 3.00,\ 2.30 + 1.90\} \tag{49}$$
$$= \min\{7.60,\ 4.20\} = 4.20 \quad \text{via edge } (3,6) \tag{50}$$

Reconstruction. Walking back from $d(6) = 4.20$:

- $d(6) = 4.20$ was achieved via edge $(3,6) \to$ est.
- $d(3) = 2.30$ was achieved via edge $(0,3) \to$ new.
- $d(0) = 0$: start.

The path is $0 \to 3 \to 6$, corresponding to the segmentation:

$$\boxed{\tau^*_{\text{newest}} = (\text{new}, \text{est})} \tag{51}$$

with total negative log-probability 4.20, or equivalently $p(\tau^*) = e^{-4.20} \approx 0.015$.

8.2 Comparison with Other Segmentations

To verify, let us compute the probability of a few alternatives:

- (new, est): weight $= 2.30 + 1.90 = 4.20$. **Best.**

- $(\text{new}, \text{es}, \text{t})$: weight $= 2.30 + 2.30 + 3.00 = 7.60$.
- $(\text{n}, \text{e}, \text{w}, \text{est})$: weight $= 3.00 + 2.53 + 2.53 + 1.90 = 9.96$.
- $(\text{n}, \text{e}, \text{w}, \text{e}, \text{s}, \text{t})$: weight $= 3.00 + 2.53 + 2.53 + 2.53 + 3.00 + 3.00 = 16.59$.

The Viterbi result (new, est) has the smallest weight, hence the largest probability, and is therefore the chosen segmentation.

9 Posterior Sampling for Subword Regularization

A key feature of UnigramLM that BPE lacks: we can **sample** from the posterior over segmentations rather than always taking the most probable one. This is called **subword regularization** [2] and acts as a form of training-time data augmentation.

Using the forward probabilities computed earlier, we can sample segmentations of "newest" according to $p(\tau \mid \text{newest})$. Typical samples (with probabilities proportional to $p(\tau)$):

- (new, est) — highest posterior, sampled most often
- $(\text{new}, \text{es}, \text{t})$ — second-tier posterior
- $(\text{n}, \text{e}, \text{w}, \text{est})$ — low posterior, sampled occasionally

This allows each training example to be tokenized differently across epochs, providing robustness against tokenization artifacts. This is impossible in BPE without ad-hoc modifications like BPE-Dropout.

10 Summary Table

Iter t	V_t	$\lvert V_t \rvert$
0	10 characters + 15 multi-char candidates	25
1	V_0 minus $\{\text{idest}, \text{we}, \text{de}, \text{id}, \text{wi}\}$	20
2	V_1 minus $\{\text{ower}, \text{ne}, \text{ow}, \text{lo}\}$	16
3	V_2 minus $\{\text{st}, \text{er}\}$	14 (target)

Table 4: Evolution of vocabulary during UnigramLM training.

The Viterbi tokenization of "newest" at the end is (new, est).

11 Comparison with BPE on the Same Corpus

It is illuminating to compare the BPE and UnigramLM trajectories on the same corpus, both with $V_{\text{target}} = 14$:

Note especially the first row: both algorithms use the pre-token frequencies c_i from $\mathcal{W}$, but they use them differently. BPE applies them as multiplicative weights on raw pair counts. UnigramLM applies them in the EM aggregation step, where they weight per-pre-token expected counts. The mathematical role is similar (frequency-weighted statistics), but the place in the algorithm is different. This explains why both algorithms produce sensible results on the same input but reach different final vocabularies.

Aspect	**BPE**	**UnigramLM**
Use of $\mathcal{W}$	Direct multiplicative weight on pair counts: $\text{freq}(a,b) = \sum_i c_i \cdot \#_i(a,b)$	Indirect: shapes seed substrings; weights EM aggregation $E[\#t] = \sum_i c_i E[\#t \mid w_i]$
Direction of growth	Bottom-up: $V_0 \to V_4$ (add 4 tokens)	Top-down: $V_{\text{init}} \to V_4$ (remove 11 tokens)
Iterations needed	4 merge iterations	3 prune iterations, each with inner EM
Final vocabulary	$V_0 \cup \{\text{es}, \text{est}, \text{ne}, \text{new}\}$	$V_0 \cup \{\text{es}, \text{est}, \text{new}, \text{low}\}$
Tokenization of "newest"	Apply rules: (new, est)	Viterbi on lattice: (new, est)
Stored at inference	V and ordered merge rules $\mathcal{R}$	V and probabilities $p(w)$
Order-dependent	Yes (rules in $\mathcal{R}$ applied in order)	No (Viterbi finds global optimum)
Supports stochastic tokenization	Not natively	Yes, via posterior sampling
Objective function	None (procedural)	Marginal likelihood $\mathcal{L}(\theta \mid V; \mathcal{W}) = \sum_i c_i \log p_\theta(w_i \mid V)$

Table 5: BPE vs. UnigramLM trained to the same target vocabulary size on the same corpus.

12 Key Observations about UnigramLM

Observation 1: Pre-token frequencies c_i enter via EM weighting. Unlike BPE, where c_i directly multiplies pair counts, UnigramLM uses c_i to weight the aggregation of per-pre-token expected counts. The corpus-level expected count is $E[\#t] = \sum_i c_i \cdot E[\#t \mid w_i]$. The mathematical effect is similar to BPE's weighting (high-frequency pre-tokens dominate), but the location in the algorithm differs.

Observation 2: Top-down structure. UnigramLM starts from a large V_{init} and prunes. The seed vocabulary is the starting point; the final vocabulary is reached by removing tokens, not adding them.

Observation 3: Probabilistic foundation. Every step has a probabilistic interpretation. EM maximizes a real likelihood. Pruning removes tokens with low likelihood contribution. Viterbi finds the maximum likelihood segmentation. This is a real generative model, not just a procedure.

Observation 4: No merge rules. There is no notion of "apply rule 1 then rule 2." The only artifacts of training are the vocabulary V and the probabilities $p(w)$. Tokenization at inference uses Viterbi on the lattice constructed from V.

Observation 5: Order independence. Reordering the tokens in V does not change tokenization. Reordering the probabilities likewise does not matter as long as each token retains its own probability. This is a fundamental property of the model: tokenization depends only on the vocabulary and probabilities, not on any procedural ordering.

Observation 6: Posterior is available. For any input string, the full posterior distribution over segmentations is computable. This enables sampling, marginalization over segmentations during training (a form of robust learning), and principled uncertainty quantification — all impossible with BPE.

Observation 7: Computational cost is higher. Training UnigramLM requires inner EM loops within each pruning iteration, making it slower than BPE for the same target vocabulary size. However, inference (Viterbi) is $O(n \cdot M)$ where n is string length and M is maximum token length, which is comparable to BPE inference complexity in practice.

Observation 8: The budget is set by V_{target}. As with BPE, the user specifies the target

vocabulary size, not the iteration count. The pruning iterations continue until $|V_t| \leq V_{\text{target}}$.

Observation 9: Inference does not use c_i. Once training is complete, Viterbi tokenizes each input independently using only V and p. The pre-token frequencies c_i shaped the probabilities during training but no longer play a role at inference time.

13 Why This Matters for the Coptic-Syriac Project

When we train SentencePiece on Coptic and Syriac corpora as described in the main tutorial, we use `model_type=unigram`. This is what XLM-R and Glot500 used in their pretraining, so our new tokenizer is consistent with the existing one.

The pre-token frequencies c_i in our Coptic + Syriac + Glot500-c sample corpus will shape:

- Which substrings end up in V_{init} (substrings of high-frequency pre-tokens are over-represented).
- Which tokens have high probability after EM convergence (tokens helpful for segmenting high-frequency pre-tokens get higher probability).
- Which tokens survive pruning (tokens with low $E[\#t] \cdot \log p(t)$ are removed first).

Specifically, the steps from the main tutorial:

- Step 2 trains UnigramLM on the mixed corpus.
- The resulting vocabulary contains tokens with associated probabilities.
- We merge the new vocabulary tokens into Glot500's tokenizer (discarding the standalone probabilities; tokenization at inference uses Glot500's joint probabilities).

The probability information that UnigramLM produces during training is genuinely useful intermediate data, even though for the project we discard it after vocabulary merging. One could in principle do more sophisticated things with it (e.g., weight new tokens by their UnigramLM probability when initializing embeddings), but the standard approach used in this tutorial just merges the vocabulary and lets the language model's subsequent training adjust everything.

The deeper takeaway: **the SentencePiece library implements UnigramLM by default, but BPE mode is also available**. When working with Coptic-Syriac (or any low-resource language), it is worth checking which mode the base model used and matching it, as we do here.

14 SimAlign Algorithm (Reference)

For completeness, we include the SimAlign pseudocode used in the main tutorial for cross-lingual alignment evaluation:

Algorithm 1 SimAlign: Word Alignment via Contextualized Embeddings

Require: Source sentence s_1, target sentence s_2, pretrained multilingual model M, layer index ℓ (default $\ell = 8$)

Ensure: Set of alignment pairs $\mathcal{A} \subseteq \{1, \ldots, |s_1|\} \times \{1, \ldots, |s_2|\}$

function SimAlign(s_1, s_2, M, ℓ)
  $\mathbf{E}_1 \leftarrow M(s_1).\text{hidden\_states}[\ell]$ ▷ Contextualized embeddings for s_1
  $\mathbf{E}_2 \leftarrow M(s_2).\text{hidden\_states}[\ell]$ ▷ Contextualized embeddings for s_2
  $\mathbf{S} \leftarrow \text{CosineSimilarity}(\mathbf{E}_1, \mathbf{E}_2)$ ▷ $S_{ij} = \cos(\mathbf{E}_1^{(i)}, \mathbf{E}_2^{(j)})$
  $\text{align}_{12}[i] \leftarrow \arg\max_j S_{ij}$ for each $i \in \{1, \ldots, |s_1|\}$
  $\text{align}_{21}[j] \leftarrow \arg\max_i S_{ij}$ for each $j \in \{1, \ldots, |s_2|\}$
  $\mathcal{A} \leftarrow \emptyset$
  for $i = 1$ to $|s_1|$ **do**
    $j \leftarrow \text{align}_{12}[i]$
    if $\text{align}_{21}[j] = i$ **then** ▷ Intersection condition
      $\mathcal{A} \leftarrow \mathcal{A} \cup \{(i, j)\}$
    end if
  end for
  return $\mathcal{A}$
end function

References

[1] Taku Kudo and John Richardson. SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, Brussels, Belgium, November 2018. Association for Computational Linguistics. DOI: 10.18653/v1/D18-2012.

[2] Taku Kudo. Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 66–75, Melbourne, Australia, July 2018. Association for Computational Linguistics. DOI: 10.18653/v1/P18-1007.

[3] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, August 2016. Association for Computational Linguistics. DOI: 10.18653/v1/P16-1162.

[4] Arthur P. Dempster, Nan M. Laird, and Donald B. Rubin. Maximum Likelihood from Incomplete Data via the EM Algorithm. *Journal of the Royal Statistical Society: Series B (Methodological)*, 39(1):1–38, 1977.